UNIVERSIDAD TECNOLÓGICA DEL PERÚ

GUÍA DE LABORATORIO 09

Autenticación con JWT en Spring Boot

De HTTP Basic a API REST stateless con tokens firmados



CONTEXTO

Semana 8: usuarios, roles, BCrypt y autenticación HTTP Basic. → Semana 9: login que emite JWT, validación de Bearer Token, autorización por roles y pruebas de seguridad.

1. Orientación del laboratorio

1.1. Logro de aprendizaje

AL FINALIZAR

El estudiante incorpora autenticación JWT a la API de productos desarrollada previamente: valida credenciales contra usuarios almacenados en MySQL, emite un token firmado y consume endpoints protegidos según los roles del usuario.

1.2. Evidencias esperadas

El endpoint POST /api/auth/login devuelve un access token cuando las credenciales son válidas.

Los endpoints GET /api/productos aceptan Bearer Token con ROLE_USER o ROLE_ADMIN.

Las operaciones de escritura requieren ROLE_ADMIN; un usuario común obtiene 403.

Las solicitudes sin token, con token alterado o vencido producen una respuesta de autenticación fallida (habitualmente 401).

Una prueba JUnit verifica la emisión y decodificación de un JWT con sus claims.

1.3. Distribución sugerida de las 3 horas

Actividad	Producto
Comprensión de JWT y flujo	Mapa conceptual
Preparación del proyecto + Maven	Proyecto compilable
Programación guiada	Login + JWT + seguridad
Postman y fallos controlados	Matriz de evidencias
JUnit, reflexión y entrega	Prueba y breve explicación

2. Antes de programar: comprender JWT

2.1. Autenticación, autorización y token

Autenticación verifica la identidad (usuario y contraseña). Autorización decide qué recursos puede utilizar la identidad autenticada. JWT (JSON Web Token) es un formato compacto para transmitir afirmaciones o claims firmados. El servidor puede verificar su integridad y vigencia sin consultar una sesión HTTP por cada solicitud.

Término	Significado práctico
JWT	Token autocontenido compuesto por header.payload.signature.
Claim sub	Identificador del usuario autenticado.
Claim exp	Fecha y hora de expiración del token.
Claim iss	Emisor del token; se valida al consumirlo.
Claim roles	Autoridades ROLE_USER y ROLE_ADMIN definidas en este laboratorio.
Bearer	Esquema de autorización HTTP que envía el token en Authorization.

IMPORTANTE

Un JWT firmado NO está cifrado: header y payload pueden leerse mediante Base64URL. La firma protege contra modificaciones no autorizadas; nunca coloques contraseñas ni datos sensibles en los claims.

2.2. Diferencias frente a HTTP Basic

Semana 8: HTTP Basic	Semana 9: JWT
Cada petición transmite credenciales codificadas para Basic.	El login recibe credenciales una vez; solicitudes posteriores llevan un token firmado.
El servidor verifica usuario y contraseña por solicitud.	El resource server verifica firma, expiración y autorización por petición.
Credenciales requieren HTTPS.	JWT también requiere HTTPS y protección contra su robo.

En esta práctica se usa HS256 (HMAC) para simplificar el laboratorio en un único servidor confiable. En ambientes empresariales con diversos servicios conviene evaluar autenticación delegada (OIDC/OAuth2) y firma asimétrica con gestión y rotación de claves.

2.3. Secuencia exacta que implementaremos

1) El cliente envía usuario y contraseña a POST /api/auth/login.

2) AuthenticationManager utiliza UserDetailsService (MySQL) y BCrypt para validar.

3) TokenService emite un JWT firmado (subject, issuer, roles, iat y exp).

4) El cliente envía Authorization: Bearer <token>.

5) Spring Security OAuth2 Resource Server verifica el JWT y convierte roles a GrantedAuthority.

6) SecurityFilterChain aplica las reglas a la URL y método HTTP.

3. Preparación: continuar el proyecto de la semana 8

3.1. Archivos que NO debes rehacer

Abre en Visual Studio Code el mismo proyecto de la Semana 8. No elimines la base de datos ni vuelvas a crear los usuarios. Verifica que los siguientes archivos y funcionalidades existan:

Elemento existente	Qué debes comprobar
model/Rol.java	Campo nombre con ROLE_USER y ROLE_ADMIN.
model/Usuario.java	username, password BCrypt, activo y relación Set<Rol>.
repository/UsuarioRepository.java	findByUsername(String) devuelve Optional<Usuario>.
security/CustomUserDetailsService.java	Convierte roles de MySQL en GrantedAuthority.
config/DatosSeguridadIniciales.java	Usuarios de laboratorio usuario y admin precargados.
controller/ProductoController.java	Recursos existentes bajo /api/productos.
MySQL: tienda_db	Tablas usuarios, roles y usuarios_roles.

Verificación SQL inicial

```
USE tienda_db;
```

```
SELECT u.username, u.activo, r.nombre AS rol
FROM usuarios u
JOIN usuarios_roles ur ON ur.usuario_id = u.id
JOIN roles r ON r.id = ur.rol_id
ORDER BY u.username;
```

CONTROL DE VERSIONES

Antes de empezar realiza un commit de la Semana 8. Modificaremos la clase SecurityConfig; si conservas el commit podrás demostrar qué cambió entre HTTP Basic y JWT.

3.2. Dependencias Maven que debemos tener

Conserva las dependencias ya instaladas: web MVC, JPA, MySQL, Security y tests. En Spring Boot 4.1.x añade el starter específico OAuth2 Resource Server (incluye el soporte de validación JWT); agrega Bean Validation para validar el request de login.

pom.xml: fragmento para agregar

```
<!-- Agregar dentro de <dependencies> -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security-oauth2-resource-server</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
<!-- Mantener spring-boot-starter-security, data-jpa,
     mysql-connector-j, webmvc y las dependencias de test -->
```

VS Code → Terminal (raíz del proyecto)

```
./mvnw clean test
# En Windows, ejecutar: mvnw.cmd clean test
```

No es necesario fijar manualmente versiones de módulos Spring Security cuando Maven hereda el BOM del parent de Spring Boot 4.1.x.

4. Configurar una clave JWT fuera del repositorio

4.1. ¿Por qué necesitamos una clave secreta?

HS256 utiliza la misma clave para generar y verificar la firma del JWT. Si alguien conoce esa clave, puede crear tokens válidos. Por ello se carga como variable de entorno; no se codifica en las clases Java ni en application.properties.

Linux y macOS

```
# Terminal Linux/macOS (misma sesión donde iniciarás Maven):
export JWT_SECRET_BASE64="$(openssl rand -base64 32)"
./mvnw spring-boot:run
```

Windows PowerShell

```
# PowerShell: generar 32 bytes criptográficamente aleatorios
$bytes = New-Object byte[] 32
$rng = [System.Security.Cryptography.RandomNumberGenerator]::Create()
$rng.GetBytes($bytes)
$env:JWT_SECRET_BASE64 = [Convert]::ToBase64String($bytes)
.\mvnw.cmd spring-boot:run
```

Si todavía no has programado el login, puedes dejar el arranque para el Paso 9; lo importante es generar la variable en la terminal que ejecutará la aplicación. Cada nuevo arranque con otra clave invalidará los tokens anteriores, lo cual es esperado en el laboratorio.

4.2. Propiedades de la aplicación

src/main/resources/application.properties

```
# Conservar las propiedades MySQL de semanas anteriores
app.jwt.issuer=tienda-api
app.jwt.ttl-minutes=30
app.jwt.secret-base64=${JWT_SECRET_BASE64}
```

COMPROBACIÓN

Si falta JWT_SECRET_BASE64, Spring Boot debe fallar al iniciar con error de resolución de configuración. No sustituyas la variable por una clave débil ni incluyas claves reales en Git.

5. Crear el emisor y verificador de JWT

5.1. Crear el paquete config/JwtConfig.java

Esta clase transforma la clave Base64 en una SecretKey; registra un JwtEncoder para emitir tokens y un JwtDecoder para comprobar la firma HS256 y el claim iss. Ambos componentes se crean una vez como beans de Spring.

src/main/java/com/utp/tienda/config/JwtConfig.java

```
package com.utp.tienda.config;

import java.util.Base64;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import com.nimbusds.jose.jwk.source.ImmutableSecret;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.*;

@Configuration
public class JwtConfig {

    @Bean
    SecretKey jwtSecretKey(@Value("${app.jwt.secret-base64}") String value) {
        byte[] raw = Base64.getDecoder().decode(value);
        if (raw.length < 32) {
            throw new IllegalArgumentException("JWT: se requieren al menos 32 bytes");
        }
        return new SecretKeySpec(raw, "HmacSHA256");
    }

    @Bean
    JwtEncoder jwtEncoder(SecretKey key) {
        return new NimbusJwtEncoder(new ImmutableSecret<>(key));
    }

    @Bean
    JwtDecoder jwtDecoder(SecretKey key,
            @Value("${app.jwt.issuer}") String issuer) {
        NimbusJwtDecoder decoder = NimbusJwtDecoder.withSecretKey(key)
                .macAlgorithm(MacAlgorithm.HS256)
                .build();
        decoder.setJwtValidator(JwtValidators.createDefaultWithIssuer(issuer));
        return decoder;
    }
}
```

5.2. Qué significa cada componente

Componente	Responsabilidad
SecretKey	Material criptográfico para HS256, cargado desde entorno.
JwtEncoder	Firma el header y los claims para producir token.
JwtDecoder	Rechaza tokens con firma, vigencia o emisor no válidos.
JwtValidators	Comprueba exp/nbf y que iss coincida con tienda-api.

6. Construir el TokenService

El servicio recibe el Authentication ya validado. NO recibe una contraseña y NO consulta MySQL para firmar; sus entradas son identidad autenticada y autoridades otorgadas por Spring Security.

src/main/java/com/utp/tienda/security/TokenService.java

```
package com.utp.tienda.security;

import java.time.Instant;
import java.time.temporal.ChronoUnit;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.*;
import org.springframework.stereotype.Service;

@Service
public class TokenService {

    private final JwtEncoder encoder;
    private final String issuer;
    private final long ttlMinutes;

    public TokenService(JwtEncoder encoder,
            @Value("${app.jwt.issuer}") String issuer,
            @Value("${app.jwt.ttl-minutes:30}") long ttlMinutes) {
        this.encoder = encoder;
        this.issuer = issuer;
        this.ttlMinutes = ttlMinutes;
    }

    public long expiresInSeconds() { return ttlMinutes * 60; }

    public String crearToken(Authentication autenticacion) {
        Instant ahora = Instant.now();
        var roles = autenticacion.getAuthorities().stream()
                .map(GrantedAuthority::getAuthority)
                .filter(a -> a.startsWith("ROLE_"))
                .toList();

        JwtClaimsSet claims = JwtClaimsSet.builder()
                .issuer(issuer)
                .subject(autenticacion.getName())
                .issuedAt(ahora)
                .expiresAt(ahora.plus(ttlMinutes, ChronoUnit.MINUTES))
                .claim("roles", roles)
                .build();

        JwsHeader header = JwsHeader.with(MacAlgorithm.HS256).build();
        return encoder.encode(JwtEncoderParameters.from(header, claims))
                .getTokenValue();
    }
}
```

PARA COMPRENDER

sub identifica al usuario; iat indica emisión; exp define el límite de validez; roles es una lista de autoridades.

La duración de 30 minutos puede ajustarse conforme a un análisis de riesgos.

7. Crear los DTOs del inicio de sesión

Usaremos records de Java para definir el contrato JSON. Esto evita exponer la entidad Usuario y su campo password almacenado. Los records también permiten validación declarativa con Jakarta Validation.

src/main/java/com/utp/tienda/dto/LoginRequest.java

```
package com.utp.tienda.dto;

import jakarta.validation.constraints.NotBlank;

public record LoginRequest(
        @NotBlank String username,
        @NotBlank String password) {}
```

src/main/java/com/utp/tienda/dto/TokenResponse.java

```
package com.utp.tienda.dto;

public record TokenResponse(
        String tokenType,
        String accessToken,
        long expiresInSeconds) {}
```

En Postman usaremos Content-Type: application/json. La entrada debe contener exactamente los campos username y password.

8. Configurar el AuthenticationManager y los filtros

8.1. Sustituir SecurityConfig.java de la Semana 8

Mantendremos BCrypt y la misma política por roles. Eliminaremos HTTP Basic, deshabilitaremos la sesión HTTP y activaremos la validación estándar del token por Resource Server. La solicitud /api/auth/login será pública, pero comprobará credenciales con AuthenticationManager.

src/main/java/com/utp/tienda/config/SecurityConfig.java

```
package com.utp.tienda.config;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.HttpMethod;
import org.springframework.security.authentication.*;
import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.oauth2.server.resource.authentication.*;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    AuthenticationManager authenticationManager(
            UserDetailsService users, PasswordEncoder passwords) {
        var provider = new DaoAuthenticationProvider(users);
```

```
        provider.setPasswordEncoder(passwords);
        return new ProviderManager(provider);
    }

    @Bean
    JwtAuthenticationConverter jwtAuthenticationConverter() {
        var authorities = new JwtGrantedAuthoritiesConverter();
        authorities.setAuthoritiesClaimName("roles");
        authorities.setAuthorityPrefix(""); // claims ya contienen ROLE_
        var converter = new JwtAuthenticationConverter();
        converter.setJwtGrantedAuthoritiesConverter(authorities);
        return converter;
    }

    @Bean
    SecurityFilterChain securityFilterChain(
            HttpSecurity http, JwtAuthenticationConverter converter)
            throws Exception {
        http
            .csrf(AbstractHttpConfigurer::disable)
            .httpBasic(AbstractHttpConfigurer::disable)
            .formLogin(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(
                    SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.POST, "/api/auth/login").permitAll()
                .requestMatchers("/api/publico/**").permitAll()
                .requestMatchers(HttpMethod.GET, "/api/productos/**")
                    .hasAnyRole("USER", "ADMIN")
                .requestMatchers(HttpMethod.POST, "/api/productos/**")
                    .hasRole("ADMIN")
                .requestMatchers(HttpMethod.PUT, "/api/productos/**")
                    .hasRole("ADMIN")
                .requestMatchers(HttpMethod.PATCH, "/api/productos/**")
                    .hasRole("ADMIN")
                .requestMatchers(HttpMethod.DELETE, "/api/productos/**")
                    .hasRole("ADMIN")
                .anyRequest().authenticated())
            .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt ->
                jwt.jwtAuthenticationConverter(converter)));
        return http.build();
    }
}
```

8.2. Explicación imprescindible

Elemento	Qué hace / por qué importa
ProviderManager + DaoAuthenticationProvider	Verifica las credenciales con UserDetailsService y BCrypt.
JwtGrantedAuthoritiesConverter	Lee el claim roles; no añade SCOPE_ porque nuestros roles ya usan ROLE_.
SessionCreationPolicy.STATELESS	No se utiliza HttpSession para conservar autenticación.
oauth2ResourceServer().jwt()	Revisa Bearer Token, firma, exp y claims antes del controller.
hasRole("ADMIN")	Exige ROLE_ADMIN: consulta las autoridades reconstruidas desde el JWT.

Deshabilitar CSRF aquí es coherente con esta API que no usa cookies de sesión para autenticación. Si posteriormente la aplicación almacena credenciales en cookies enviadas automáticamente por el navegador, hay que reevaluar la protección CSRF.

9. Implementar AuthController: emitir un token

src/main/java/com/utp/tienda/controller/AuthController.java

```
package com.utp.tienda.controller;

import com.utp.tienda.dto.LoginRequest;
import com.utp.tienda.dto.TokenResponse;
import com.utp.tienda.security.TokenService;
import jakarta.validation.Valid;
```

```
import java.util.Map;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.*;
import org.springframework.security.core.Authentication;
import org.springframework.security.core.AuthenticationException;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.oauth2.server.resource.authentication.JwtAuthenticationToken;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager manager;
    private final TokenService tokens;

    public AuthController(AuthenticationManager manager, TokenService tokens) {
        this.manager = manager;
        this.tokens = tokens;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@Valid @RequestBody LoginRequest request) {
        try {
            Authentication autenticacion = manager.authenticate(
                UsernamePasswordAuthenticationToken.unauthenticated(
                        request.username(), request.password()));
            String jwt = tokens.crearToken(autenticacion);
            return ResponseEntity.ok(new TokenResponse(
                    "Bearer", jwt, tokens.expiresInSeconds()));
        } catch (AuthenticationException ex) {
            return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
                    .body(Map.of("error", "Credenciales inválidas"));
        }
    }

    @GetMapping("/me")
    public Map<String, Object> miPerfil(JwtAuthenticationToken auth) {
        var roles = auth.getAuthorities().stream()
            .map(GrantedAuthority::getAuthority)
            .filter(a -> a.startsWith("ROLE_"))
            .toList();
        return Map.of("username", auth.getName(), "roles", roles);
    }
}
```

Observa que el login nunca devuelve la contraseña ni el hash. El endpoint /api/auth/me obtiene el usuario de la autenticación JWT resultante, no vuelve a ejecutar un login.

9.1. Estructura final de seguridad

Estructura de paquetes

```
src/main/java/com/utp/tienda/
  config/      JwtConfig.java  SecurityConfig.java
               DatosSeguridadIniciales.java (sin cambios)
  controller/  AuthController.java  ProductoController.java
               PublicoController.java (sin cambios)
  dto/         LoginRequest.java    TokenResponse.java
  model/       Usuario.java         Rol.java (sin cambios)
  repository/  UsuarioRepository.java RolRepository.java
  security/    CustomUserDetailsService.java (sin cambios)
               TokenService.java
```

10. Ejecutar y verificar por etapas

10.1. Arrancar backend

Detén la aplicación de la semana anterior. Abre una terminal nueva en VS Code, establece JWT_SECRET_BASE64 y arranca con Maven. Revisa que MySQL esté activo y que no haya otro proceso utilizando el puerto 8080.

Comandos ejecutados en la misma sesión con la variable JWT_SECRET_BASE64

```
./mvnw clean test
./mvnw spring-boot:run
```

10.2. Probar endpoint público

Postman - sin autenticación

```
GET http://localhost:8080/api/publico/estado

Esperado: HTTP 200
{"estado":"API disponible"}
```

10.3. Iniciar sesión como usuario

Postman: Body → raw → JSON

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{
  "username": "usuario",
  "password": "Usuario123*"
}
```

Ejemplo de respuesta (token ilustrativo)

```
HTTP/1.1 200 OK
{
  "tokenType": "Bearer",
  "accessToken": "eyJ...<cadena abreviada>...",
  "expiresInSeconds": 1800
}
```

Copia el valor completo de accessToken. En Postman selecciona Authorization → Type: Bearer Token → pega el valor SIN escribir nuevamente la palabra Bearer en el campo Token.

10.4. Consumir un recurso protegido

Postman → Authorization → Bearer Token

```
GET http://localhost:8080/api/productos
Authorization: Bearer <TOKEN_USUARIO>
```

El usuario con ROLE_USER puede consultar productos. Intenta registrar un producto con POST usando el mismo token: la respuesta debe ser 403 Forbidden porque la política exige ROLE_ADMIN.

10.5. Iniciar sesión como administrador

Postman: obtener nuevo token

```
POST http://localhost:8080/api/auth/login
Content-Type: application/json

{"username":"admin","password":"Admin123*"}
```

Postman: POST admin (el contrato de producto es el de la semana 7)

```
POST http://localhost:8080/api/productos
Authorization: Bearer <TOKEN_ADMIN>
Content-Type: application/json

{
```

```
  "nombre":"Audífonos USB",
  "categoria":"Tecnologia",
  "precio":190.00,
  "stock":7
}
```

El código de éxito del POST puede ser 201 Created o 200 OK según la implementación conservada de ProductoController. Para este laboratorio no alteramos la lógica de productos.

11. Matriz de pruebas y resultados esperados

N°	Solicitud	Token	Estado esperado
01	GET /api/publico/estado	Ninguno	200
02	POST /api/auth/login (credenciales válidas)	Ninguno	200 + JWT
03	POST /api/auth/login (contraseña errónea)	Ninguno	401
04	GET /api/productos	Ninguno	401
05	GET /api/productos	USER válido	200
06	POST /api/productos	USER válido	403
07	POST /api/productos	ADMIN válido	200 o 201
08	GET /api/auth/me	ADMIN válido	200 + roles
09	GET /api/productos	JWT modificado	401
10	GET /api/productos	JWT vencido	401

11.1. Forzar fallo de firma

Copia un token válido, cambia un carácter de su tercer segmento (firma) y vuelve a enviarlo. No modifiques el original guardado. Spring Security debe rechazarlo antes de acceder a ProductoController. Si cambias un carácter en payload también fallará la firma.

11.2. Comprobar la expiración sin esperar 30 minutos

En application.properties usa temporalmente app.jwt.ttl-minutes=1, reinicia la aplicación y solicita un token nuevo; espera más de un minuto y comprueba la respuesta. Nota técnica: algunas configuraciones del decodificador permiten una tolerancia de reloj (clock skew), por lo que la respuesta puede tardar más de 60 segundos en cambiar a 401. Para una prueba estricta se puede configurar JwtTimestampValidator con tolerancia cero en un perfil de test.

11.3. Examinar un JWT sin revelar información sensible

En un decodificador JWT local o mediante una utilidad que trabaje sin enviar datos a Internet, inspecciona el header y payload. Debes observar alg=HS256, sub, iss, iat, exp y roles. No publiques claves, contraseñas o JWT auténticos en capturas compartidas.

12. Prueba automatizada JUnit: generación y verificación

La prueba siguiente comprueba el token sin arrancar MySQL: construye una clave exclusiva de test, emite un JWT y verifica que el subject y los roles sobreviven al proceso de codificación y decodificación. No reutilices la clave de prueba en otros entornos.

src/test/java/com/utp/tienda/security/TokenServiceTest.java

```
package com.utp.tienda.security;

import java.nio.charset.StandardCharsets;
import java.util.List;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import com.nimbusds.jose.jwk.source.ImmutableSecret;
import org.junit.jupiter.api.Test;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.oauth2.jose.jws.MacAlgorithm;
import org.springframework.security.oauth2.jwt.*;

import static org.junit.jupiter.api.Assertions.*;

class TokenServiceTest {
    @Test
    void tokenContieneUsuarioYRolFirmados() {
        byte[] raw = "0123456789abcdef0123456789abcdef"
                .getBytes(StandardCharsets.UTF_8);
        SecretKey key = new SecretKeySpec(raw, "HmacSHA256");
        JwtEncoder encoder = new NimbusJwtEncoder(new ImmutableSecret<>(key));
        JwtDecoder decoder = NimbusJwtDecoder.withSecretKey(key)
                .macAlgorithm(MacAlgorithm.HS256).build();
        TokenService service = new TokenService(encoder, "tienda-api", 30);

        var auth = UsernamePasswordAuthenticationToken.authenticated(
                "admin", null,
                List.of(new SimpleGrantedAuthority("ROLE_ADMIN")));
        String token = service.crearToken(auth);
        Jwt decoded = decoder.decode(token);

        assertEquals("admin", decoded.getSubject());
        assertEquals("tienda-api", decoded.getIssuer().toString());
        assertTrue(decoded.getClaimAsStringList("roles")
                .contains("ROLE_ADMIN"));
        assertNotNull(decoded.getExpiresAt());
    }
}
```

Ejecución de prueba

```
./mvnw -Dtest=TokenServiceTest test
# O usa Run Test directamente encima del método en VS Code.
```

Si la prueba no se ejecuta, revisa el package, la carpeta src/test/java y que exista la dependencia de test agregada en semanas anteriores.

13. Diagnóstico de incidencias frecuentes

Síntoma	Causa probable	Acción concreta
Falló el arranque: JWT_SECRET_BASE64	Variable ausente o terminal distinta.	Exporta la variable en la misma terminal del Maven.
IllegalArgumentException: 32 bytes	Clave HMAC insuficiente.	Genera 32 bytes con un CSPRNG, codifica Base64.
POST /login = 401	Credenciales, BCrypt o usuario deshabilitado.	Verifica usuarios en MySQL; no reinsertes contraseñas en claro.
GET con JWT = 401	Token sin Bearer, firma, iss o exp incorrectos.	Verifica header, issuer y clave de arranque.
USER puede leer pero POST = 403	Acceso correctamente restringido.	Usa JWT de admin para operaciones de escritura.
ADMIN recibe 403	Claim roles mal formado o prefijo duplicado.	Roles deben ser ROLE_ADMIN; converter con prefijo vacío.

Síntoma	Causa probable	Acción concreta
Compilación de starter	Nombre antiguo del starter Boot 4.	Usa starter-security-oauth2-resource-server.
Token anterior deja de funcionar	Se cambió JWT_SECRET_BASE64.	Inicia sesión otra vez; clave de firma cambió.

14. Seguridad profesional: límites de este laboratorio

HTTPS es obligatorio cuando hay usuarios reales: JWT no evita la interceptación si se transmite por HTTP público.

Evita tokens en query string, logs, capturas, historiales del navegador o repositorios Git.

No almacenar contraseñas en JWT; no usar JWT como reemplazo de validaciones de negocio.

La desactivación o cambio de rol en MySQL no invalida automáticamente un JWT emitido: sus claims permanecen vigentes hasta exp salvo que implementes revocación/introspección u otra estrategia.

Una API distribuida debe evaluar RS256/ES256, JWK, rotación de claves y un proveedor de identidad; HS256 aquí es un recurso formativo de una sola aplicación.

Para Angular, evitar guardar tokens de larga vida en localStorage; evaluar riesgos XSS, estrategia de sesión y almacenamiento adecuado.

Una aplicación con cookies debe volver a analizar CSRF: stateless no significa inmunidad automática a CSRF en todos los diseños.

EXTENSIÓN OPCIONAL

Analiza por qué un token con ROLE_ADMIN previamente emitido puede seguir autorizando incluso si después retiramos ese rol en MySQL. Propón mitigaciones: expiraciones breves, revocación por identificador jti, versiones de sesión o consulta de permisos sensibles en cada solicitud.

15. Actividad integradora y evidencias

15.1. Reto de aplicación

Desarrolla un endpoint GET /api/admin/reporte que devuelva un JSON con la fecha del servidor y el total de productos. Protege el endpoint para ROLE_ADMIN. Verifica 401 sin token, 403 con USER y 200 con ADMIN. Conserva la arquitectura Controller → Service → Repository existente.

16. Síntesis de aprendizaje y siguiente hito

CAPACIDAD LOGRADA

La API pasa de autenticar usuarios con Basic a emitir tokens JWT y verificar permisos sin crear sesiones HTTP. Se mantienen la persistencia MySQL y la separación por capas, demostrando que la capa de seguridad puede evolucionar sin rehacer el dominio de productos.

17. Referencias y lecturas técnicas

Spring Security 7.1.1: OAuth2 Resource Server JWT — https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html

Spring Security 7.1.1: DaoAuthenticationProvider — https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/dao-authentication-provider.html

Spring Boot 4.1.x: referencia Web MVC — https://docs.spring.io/spring-boot/reference/web/index.html

RFC 7519: JSON Web Token (JWT) — https://www.rfc-editor.org/rfc/rfc7519

OWASP JWT Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html